Roopam Bag Mall

Project Roadmap and Oral Exam Guide

A practical study guide to the retail system and 3D custom bag request platform

Prepared for project explanation and viva practice | 23 October 2026

What you need to understand

Learn the project as a set of connected workflows: a customer designs a bag, staff receive stock, a cashier makes a sale, and an administrator reviews requests. Then learn which code and libraries make each workflow possible. You should be able to explain one action from the screen to the database and back.

A short introduction you can say aloud

“Our project is Roopam Bag Mall, a full stack application for managing a bag retail store and collecting custom bag design requests. Staff manage products, suppliers, purchases, stock movements, counter sales and reports. Customers can register, customize a bag through an interactive 3D preview and track the store’s response. React and TypeScript provide the interface, Spring Boot provides secured REST APIs, and PostgreSQL stores the business data. The server enforces roles, validates operations and maintains stock history.”

Keep the two business flows clear

Retail operations	Custom bag requests
Purchase goods from a supplier, receive stock, sell at the counter and record payments.	Choose a design, submit a request and receive an administrator’s review.
Purchases and sales affect product stock and create inventory ledger entries.	Submitting a request does not reserve stock, create a retail order or take payment.

How to use this guide

Start with the roadmap on page 2. Read pages 3–11 while opening the named code files. Use page 12 to prepare the local demo. Practise the questions on pages 13–14 aloud, then use page 15 as your final revision sheet.

Aim to explain the reason behind a library or rule, not just remember its name. Use the numerical examples when an examiner asks you to demonstrate understanding.

1 Study roadmap

Suggested pace: seven study sessions of about 60–90 minutes. This is a preparation plan, not a record of completed work. Finish each session by explaining its checkpoint without reading.

Session	Learn and inspect	Checkpoint
1 Project and basics	Read README.md. Learn browser, server, database, HTTP, JSON and CRUD. Identify the users and modules.	Give the project introduction and explain a sale versus a custom request.
2 Interface and data	Learn React components, props, state, hooks, TypeScript types and async/await. Read App.tsx and useProducts.ts.	Explain how a product list loads and refreshes after creation.
3 Backend layers	Learn Java classes, interfaces, enums, exceptions and dependency injection. Trace ProductController through service and repository.	Explain controller, service, DTO, entity and repository separately.
4 Database and stock	Read Product.java, ProductRepository.java and the sale/purchase services. Learn relationships, BigDecimal and transactions.	Calculate stock changes and explain why stock cannot be edited through the product form.
5 Security	Read SecurityConfig, JwtTokenService, JwtAuthenticationFilter, AuthContext and axiosClient.	Trace login and explain 401, 403, ownership and server role checks.
6 Custom bag feature	Read BagPreview3D, bagConfiguration, CustomBagRequestServiceImpl and CustomBagPricingService.	Explain configuration to 3D preview, submission, price calculation and review status.
7 Rehearsal	Follow the demo on page 15. Review the tests and the limitations. Answer pages 13–14 without notes.	Deliver a five minute explanation and show a successful operation plus a rejected operation.

If the oral is tomorrow

Prioritize the introduction, architecture on page 5, security on page 7, sale flow on page 8 and custom request flow on pages 10–11. Then practise the twenty questions. Know the exact boundaries of implemented features before discussing future improvements.

A simple self assessment

For each topic, score yourself 0 if you only recognize the term, 1 if you can explain it, and 2 if you can locate it in the code and demonstrate it. Revisit topics below 2 before memorizing additional library APIs.

2 Frontend libraries to know

These dependencies are declared in frontend/package.json. Version families below describe this repository; they are not claims about the newest releases. Learn the concepts in the third column before studying advanced APIs.

Library or tool	Purpose in this project	What to learn first
React 19 and React DOM	Build and mount the customer and staff interfaces as components.	Components, JSX, props, useState, useEffect, context and event handlers.
TypeScript 6	Describe request, response and configuration shapes during development.	Interfaces, unions, optional fields and type checking. Types do not validate runtime HTTP data.
React Router DOM 7	Map URLs to screens and support protected role routes.	Routes, navigation, route parameters, redirects and nested layouts.
TanStack React Query 5	Load server data, cache query results and coordinate mutations.	useQuery, useMutation, queryKey, loading/error states and invalidateQueries.
Axios 1	Send HTTP requests through the shared /api/v1 client.	GET/POST/PUT/PATCH, request bodies, responses, interceptors and async errors.
Three.js 0.186	Provide 3D geometry, materials, textures, cameras and rendering.	Scene, mesh, geometry, material, lighting and camera.
React Three Fiber 9	Describe the Three.js scene using React components.	Canvas and how configuration props update the selected model.
Drei 10	Supply reusable 3D helpers, including OrbitControls and ContactShadows.	Rotation/zoom controls and scene helpers.
Tailwind CSS 4	Style layouts, spacing, colors and responsive screens.	Utility classes and responsive variants.
Lucide React	Provide interface icons.	Using an icon component with size and accessible surrounding labels.
Vite 8 and React plugin	Run the development server, proxy API calls and build assets.	Dev server, production build and /api proxy configuration.
Oxlint	Check source code for lint problems.	What npm run lint checks and how it differs from a test.

Study priority

First: React, TypeScript, Router, Query and Axios. Next: the three 3D libraries, because the designer is a distinctive feature. Finally: styling, icons and build tooling. Node.js runs frontend tools; the application’s business backend is Java.

3 Backend libraries and annotations

Dependency or platform	What it does	What you should explain
Java 21 and Maven	Java runs backend code; Maven resolves dependencies and builds/tests it.	Classes, interfaces, collections, exceptions, streams and pom.xml.
Spring Boot 3.3.3	Starts and configures the backend application.	Auto configuration, component scanning and constructor dependency injection.
Spring Web	Maps HTTP routes, reads request data and writes JSON responses.	REST controllers, HTTP methods and ResponseEntity. JSON serialization is provided through the web stack.
Spring Data JPA and Hibernate	Map entities to tables and implement repository persistence.	JPA is the persistence API; Hibernate is its implementation. Repositories still execute database operations.
Jakarta Validation	Validate annotated request fields.	Constraints such as required fields and positive quantities; @Valid triggers validation at the boundary.
Spring Security and BCrypt	Authorize API access and hash passwords.	Authentication versus authorization; BCrypt is hashing rather than reversible encryption.
Spring Security OAuth2 JOSE	Provide JWT encoding/decoding through Nimbus support.	Signed HS256 tokens, claims, expiration and signature checking. JWT is a token format, not a separate database.
PostgreSQL JDBC driver	Connect Java persistence to PostgreSQL.	Database URL, credentials and the role of a JDBC driver.
Flyway core and PostgreSQL module	Apply ordered SQL schema migrations.	V1–V11 migrations; Hibernate uses ddl-auto=validate rather than owning schema changes.
Spring Boot Test and Security Test	Supply backend testing support.	JUnit assertions, Mockito mocks, MockMvc requests and authorization scenarios.

Annotations you should recognize

@RestController exposes HTTP handlers; @RequestMapping sets a route prefix; @GetMapping and @PostMapping choose operations. @Service marks business logic. @Entity maps a persistent class; @Id identifies its primary key; @ManyToOne and @OneToMany describe relationships. @Transactional groups database work. @RestControllerAdvice handles errors consistently.

Why constructor injection matters

A service receives repositories and collaborators through its constructor. Spring provides those objects. The service does not construct its own database access objects, which makes dependencies explicit and allows tests to substitute mocks.

4 Architecture and one full request

The system has a React frontend, a Spring Boot backend and a PostgreSQL database. The frontend displays information and collects input. The backend owns authorization and business rules. The database stores records and enforces relational constraints.

Direction	What happens
Screen → React hook	A form or page calls a feature hook such as useProducts.
Hook → Axios → API	Axios sends HTTP to /api/v1 and attaches a bearer JWT when available.
Security → Controller	The filter checks identity; security rules allow or reject the requested operation. The controller maps and validates input.
Controller → Service	The service checks business rules and coordinates the operation.
Service → Repository → PostgreSQL	JPA/Hibernate execute reads and writes through repositories, inside a transaction where required.
Database → DTO → Screen	The service returns a response DTO as JSON. React Query updates or refetches the cached data, and React renders the result.

Example of creating a product

An authorized inventory manager submits the product form. The create mutation in useProducts.ts posts a ProductRequest to /products. ProductController delegates to the product service, which checks catalog rules and duplicate identifiers before saving a Product through ProductRepository. The response becomes a ProductResponse. The hook invalidates the products query key so a list query can fetch fresh data.

What each object means

A request DTO describes accepted input; a response DTO describes returned data. An entity describes persistent state and relationships. Keeping DTOs separate allows the API to expose a deliberate data shape and avoid accepting internal fields just because they exist in the database.

Local browser and API connection

During development the browser uses port 5173 and Spring Boot uses port 8080. Vite forwards /api calls to the backend. Production hosting needs an equivalent API routing setup. The browser does not connect directly to PostgreSQL.

Be ready to point to these files

Frontend: src/api/axiosClient.ts and src/features/products/useProducts.ts under frontend. Backend: controller/ProductController.java, service/impl/ProductServiceImpl.java and repository/ProductRepository.java under the Java package.

5 Database model and data integrity

Think in records and relationships. A product is reused in many transactions; each sale or purchase stores its own item quantities and prices. This keeps individual business events understandable after catalog data changes.

Relationship	Meaning
Category 1 → many Products	Every product belongs to a category.
Brand 1 → many Products	A product can reference a brand; its brand relationship is optional.
Supplier 1 → many Purchases	Purchases identify the supplier who supplied the goods.
Purchase 1 → many PurchaseItems	Each line references a product, quantity and purchase price.
Purchase 1 → many PurchasePayments	Supplier payment records can be added over time.
Sale 1 → many SaleItems	A counter sale contains product lines with selling prices and quantities.
Sale 1 → 1 SalePayment	The sale model has a single linked payment record.
Product 1 → many InventoryTransactions	Each movement records before/after quantity and a reference.
Sale 1 → many SaleAuditHistory entries	Sale creation, editing and cancellation can be traced.
AppUser 1 → many CustomBagRequests	A request belongs to its authenticated customer.

Keys and constraints

A primary key identifies one row. A foreign key links related rows and prevents invalid references. Unique constraints protect identifiers such as SKU and barcode; the project also has a migration for case insensitive product identifiers. Input validation helps users, while database constraints remain a second line of protection.

Transactions and simultaneous sales

A sale changes several records: sale, items, payment, stock, movement history and audit. A database transaction makes those related writes commit together or roll back on an applicable failure. ProductRepository.findByIdForUpdate uses PESSIMISTIC_WRITE. The sale service locks product rows in sorted ID order before checking stock, reducing conflicting updates and helping avoid deadlocks. This does not prove every possible concurrency scenario is solved.

Schema and file storage

Flyway applies ordered SQL migrations V1–V11. Hibernate validates the mapped schema. Custom logo bytes are stored as private files; the database stores a logo reference and the design configuration. A backup or demo transfer therefore needs both the database and uploaded files.

6 Login roles and ownership

Trace login in six steps

- The user posts login credentials to /api/v1/auth/login. The authentication service checks the stored account and BCrypt password hash.
- JwtTokenService issues an HS256 signed token containing email as subject, user ID, role, issue time and expiry. The default configured lifetime is eight hours.
- AuthContext stores the token through authStorage.ts in localStorage and holds the user in React context.
- The Axios request interceptor sends Authorization: Bearer <token> on subsequent requests.
- JwtAuthenticationFilter verifies the token and checks that the database user is active and still matches the token email and role. Spring Security then applies endpoint role rules.
- A 401 response triggers session clearing and clears the React Query cache. Logout removes the local token; this implementation does not demonstrate a token revocation list.

Role	Main permitted operations
Anonymous	Public storefront and authentication endpoints.
CUSTOMER	Submit custom requests and read own requests and logos.
CASHIER	Staff dashboard, read product information, POS, sale history and sale editing.
INVENTORY_MANAGER	Retail stock/catalog/suppliers/purchases, reports and sale cancellation, plus staff sales access.
ADMIN	Staff capabilities and administrator custom request review.

Ownership is more than having the customer role

The server derives the customer ID from the authenticated identity. A customer detail lookup uses both request ID and customer ID. Guessing another customer's request ID returns 404 rather than exposing that record. The administrator has a separate review API. Uploaded logos use owner/admin authenticated access.

Short answers for security follow ups

401 means authentication is missing, invalid or expired; 403 means the authenticated role is forbidden. Frontend route guards improve navigation, but backend rules enforce permissions. A signed JWT is not encrypted, so its payload must not contain secrets. localStorage persistence is convenient but exposes the token to successful script injection; XSS prevention and secure deployment remain important.

7 How a counter sale works

From POS to saved sale

- The cashier selects products, quantities, selling prices, a discount and a payment method, then submits a SaleRequest.
- SaleServiceImpl checks the date and items. Duplicate lines for a product are merged only when their rounded selling prices agree.
- The service locks the involved products in sorted ID order, rejects inactive products and verifies sufficient stock.
- It calculates line totals and the subtotal using BigDecimal, applies one discount mode and calculates the grand total. A positive total requires a payment method.
- It creates a COMPLETED sale with item records and one payment record, reduces product stock and adds STOCK_OUT ledger entries referenced to the sale.
- It records an audit entry using the authenticated actor and returns a SaleResponse. These database operations are coordinated by the service transaction.

A numerical example you can explain

Calculation	Result
Two backpacks at INR 800 each	Subtotal = INR 1600.00
Discount of 10 percent	Discount = INR 160.00
Subtotal less discount	Grand total = INR 1440.00
Stock before 10, quantity sold 2	Stock after = 8; ledger records 10 → 8

The current sale service calculates grand total as subtotal minus discount. It does not add a sale tax calculation. Purchase tax is handled separately. A percentage discount and a fixed discount cannot both be supplied in the same sale request.

Editing and cancelling

Editing applies quantity differences. If the original sale used 2 units and the edited sale uses 3, only one additional unit leaves stock. The service also adjusts totals/payment and writes audit history. Cancellation is restricted to inventory managers and administrators, restores sold stock, records a SALE_CANCEL movement and stores the actor and reason. A cancelled sale is retained for history.

Why BigDecimal and an audit trail

BigDecimal supports explicit decimal calculations and rounding for money. The ledger explains stock changes; sale audit history explains who performed a sale action. They serve related but different questions. Recording a payment method is not evidence of an external payment gateway transaction.

8 Purchases inventory and reports

Purchasing from a supplier

A purchase contains a supplier, item quantities and purchase prices, discount information, tax information and optional supplier payments. PurchaseServiceImpl validates the inputs, calculates totals, saves the purchase and increases stock with STOCK_IN movements. The purchase is an inward retail event, whereas the customer sale is an outward event.

Example calculation	Value
Five units at INR 500	Subtotal = INR 2500.00
Discount INR 100	Taxable amount = INR 2400.00
Example configured tax rate 5 percent	Tax = INR 120.00
Taxable amount plus tax	Grand total = INR 2520.00
Supplier payment INR 1000	Remaining amount = INR 1520.00

The tax percentage above is only an arithmetic example, not a statement about an applicable legal tax rate. The service rejects supplier payments that exceed the purchase total. It also prevents reducing an edited total below the amount already paid.

Editing or cancelling a purchase

Edits apply the difference between old and new quantities and produce corresponding movements. Cancellation removes the stock introduced by that purchase. If subsequent activity leaves insufficient stock to reverse the receipt, the operation must be rejected rather than allowing a negative balance. Supplier payment records can be managed through purchase payment endpoints.

Why inventory has its own ledger

The product stock field answers “how many are available now?” InventoryTransaction answers “how did the quantity change?” It stores product, type, quantity, before/after balance, reference, movement date and reason. Catalog editing does not directly change stock. Explicit inventory operations and purchase/sale flows provide traceable movements.

Understand report totals before comparing them

Reports cover daily and date range sales, product/category performance, payment methods and cancelled sales. Product and category amounts represent gross item amounts before sale level discounts. Therefore they need not equal a net grand total report. Completed and cancelled sales have different reporting semantics. Sales revenue alone is not a verified net profit calculation.

9 How the 3D designer works

Configuration is the source of the preview

BagConfiguration is a TypeScript interface containing bag type, size, material, colors, pocket choices, compartment count, padding, water resistance, logo and text settings. UI controls update configuration state. The preview receives that configuration as props, so changing a control updates the rendered model.

Stage	Implementation idea
Choose a template	BACKPACK, LAPTOP_BAG or DUFFEL_BAG selects the corresponding model component.
Set appearance	Colors map to material colors; size maps to scale; options control supported model parts.
Render the scene	React Three Fiber Canvas hosts the Three.js scene and camera.
Make it readable	Lights illuminate the model and ContactShadows ground it visually.
Inspect it	Drei OrbitControls lets the user rotate and zoom within configured limits.
Handle unsupported devices	A scene error boundary displays a message if the 3D preview cannot start, for example when WebGL is unavailable.

The three libraries have different jobs

Three.js supplies the underlying graphics objects and rendering capabilities. React Three Fiber connects those capabilities to React's component model. Drei provides ready made helpers. In an oral answer, say how these layers work together rather than describing all three as interchangeable 3D libraries.

What is stored for later viewing

The backend stores design choices and a logo reference. When the saved request is opened, those choices can be used to reconstruct the preview. This is a configurable product visualization; it does not demonstrate automatic generation of manufacturing CAD files or a production ready pattern.

Rules depend on bag type

The server permits 1–4 compartments for Backpack and Duffel Bag, and 1–3 for Laptop Bag. Laptop Bag does not support side pockets; Duffel Bag does not support laptop padding. Both interface restrictions and backend validation matter because a caller can bypass the interface.

Open these frontend files together

Under frontend/src/features/custom-bag/three: CustomBagDesigner.tsx, bagConfiguration.ts, bagTemplates.ts, BagPreview3D.tsx, ClassicBackpackModel.tsx, LaptopBagModel.tsx and DuffelBagModel.tsx. Follow a single color change from its control to the model material.

10 Custom request submission and review

The submission journey

A signed in customer reviews a standard design and submits multipart form data. The part named request contains JSON and the optional part named logo contains an image. The backend validates the model choices, derives ownership from the authenticated customer, stores the logo, recalculates the estimate and saves a SUBMITTED request. It does not accept a customer supplied owner or final price as authoritative.

Server pricing rules in INR

Component	Added estimate
Base bag	Backpack 1200; Laptop Bag 1400; Duffel Bag 1600
Size	Small 0; Medium 150; Large 300
Material	Polyester 0; Canvas 150; Leather 500
Compartments	100 for each compartment after the first
Options	Front pocket 100; side pockets 150; laptop padding 200; water resistance 150
Branding	Uploaded logo 200; nonempty custom text 100

Example: a medium canvas Backpack with two compartments, a front pocket, no side pockets, no padding, no water resistance, no logo and no text costs 1200 + 150 + 150 + 100 + 100 = INR 1700.00. The figure is an estimate for store review. Color does not add a surcharge in the pricing service.

Allowed review transitions

```
SUBMITTED  →  REVIEWING  →  APPROVED  →  COMPLETED
    ↓            ↓
 REJECTED     REJECTED
```

Only ADMIN can review requests. Rejection is allowed from SUBMITTED or REVIEWING. APPROVED can move to COMPLETED. COMPLETED and REJECTED are terminal. CANCELLED exists in the status enum but is not an available transition in this review map. A direct SUBMITTED to APPROVED update is rejected.

Logo validation and privacy

The server accepts PNG/JPEG images up to 2 MB, checks declared type and file signature, generates a UUID filename and restricts storage paths. The database stores the file reference. Owner/admin APIs stream logos after access checks. If request persistence throws inside the create flow after storing the file, the service attempts to delete that file.

Special design requests are deferred. Approval does not automatically create a sale, reserve stock, collect a payment or start a manufacturing integration.

11 Running testing and honest limitations

Prepare the local demo

Use Java 21, Maven 3.9+, PostgreSQL and a Node.js version compatible with the repository's Vite requirements. Create the inventory_management database and configure the DB environment variables as needed. Read local configuration for development accounts; do not put passwords in your presentation.

```
mvn spring-boot:run "-Dspring-boot.run.profiles=dev"

# In a second terminal, from the project root
cd frontend
npm ci
npm run dev
```

Open http://localhost:5173. The backend listens on port 8080. Flyway runs pending migrations on backend startup. Preserve existing data and uploads. The dev profile inserts missing staff accounts; customers register through the application.

Verification commands to understand

```
mvn test

# From the frontend directory
npm run lint
npm run build
```

The backend includes service, controller, security and entity tests. Examples cover sale/purchase business rules, JWT behavior, ownership and authorization. Separate opt in PostgreSQL regression suites exercise actual migrations and database behavior; see README.md for their system properties. Commands here are study instructions, not a claim that these checks were run while preparing this guide.

What each kind of check proves

A unit test isolates business logic with collaborators mocked. A controller test checks an HTTP contract and validation/security behavior in its configured test scope. A PostgreSQL regression test checks real database behavior. Lint and a successful frontend build catch a different class of defects; neither proves the full application works in a browser.

Error handling to recognize

GlobalExceptionHandler returns structured errors: 400 for invalid input, 404 for missing resources and 409 for duplicate resources, with a generic 500 response for unexpected errors. Security handlers return 401 or 403. The interface should show the user a useful error and avoid claiming success after a failed API request.

Discuss improvements as future work

Examples include payment gateway integration, conversion of approved custom requests into fulfilment/orders, stronger token lifecycle handling, upload backup/object storage, and pagination for growing datasets. Admin request filtering currently loads records and filters in Java. Do not claim production scale, automated manufacturing or measured performance without evidence.

12 Oral practice questions on the core system

1 What problem does the project solve

It brings retail catalog, suppliers, stock, purchases and counter sales into one system, while giving customers a way to visualize and submit custom bag designs. Its benefit is connected records and traceable workflows; any claimed measured time saving would need actual evaluation.

2 Why use React and TypeScript

React builds reusable interactive screens from components. TypeScript describes shapes such as ProductResponse and BagConfiguration and catches type mismatches during development. The backend must still validate requests at runtime.

3 Why use Spring Boot

It provides application startup, configuration and integration with web, security and persistence components. Controllers, services and repositories make the backend responsibilities easier to organize and test.

4 What is a REST API in your project

It is the HTTP contract between the interface and backend. For example, GET /api/v1/products reads products, POST creates one, PUT updates one and PATCH can change its status. JSON carries most business data; logo submission uses multipart.

5 What is the difference between entity and DTO

An entity represents database mapped state and relationships. A DTO represents the input/output contract of an operation. For example, Product is an entity and ProductRequest/ProductResponse are API DTOs.

6 Why do you need both Axios and React Query

Axios performs the HTTP exchange and attaches the token through an interceptor. React Query manages the lifecycle of server data, including loading, errors, cache keys and invalidation after a mutation.

7 How is stock kept consistent during a sale

The service locks product rows, checks stock, calculates the sale, decreases stock and writes movement/audit records in a transaction. Stock validation happens on the server, because the browser's displayed quantity may be stale.

8 What happens if two cashiers sell the last item

The pessimistic product lock makes conflicting operations wait on the same row. A later operation should observe the updated stock and fail if the quantity is no longer available. I would demonstrate this with a concurrency regression test before making stronger guarantees.

9 Why use Flyway when Hibernate is present

Flyway controls ordered schema changes through SQL migrations. Hibernate maps Java entities and is configured to validate the schema. The two tools handle different parts of database persistence.

10 What happens when a sale is cancelled

An authorized manager or admin supplies a reason. The service retains the sale as CANCELLED, restores item quantities, creates reversal ledger entries and records the authenticated actor and cancellation details.

13 Oral practice questions on design and security

11 How does the 3D designer update immediately

User controls update configuration state. React passes that configuration to the selected bag model inside a React Three Fiber Canvas. Model parts, material colors and scale reflect the configuration; Drei controls provide rotation and zoom.

12 Is this artificial intelligence or a CAD system

The implemented feature is configuration driven 3D visualization. The inspected flow does not demonstrate an AI model or manufacturing CAD export. The bag choices drive predefined model components and pricing rules.

13 Why calculate the estimate on both sides

The frontend estimate provides immediate feedback. The server recalculates from validated options so changing the browser payload cannot set an arbitrary trusted estimate. The stored result remains an estimate for store review.

14 Does submitting a custom bag purchase it

No. Submission creates a design request linked to the customer. The administrator reviews its status and adds a note. This flow does not reserve retail stock or accept a payment.

15 How are passwords and tokens protected

Passwords are stored as BCrypt hashes. JWTs are signed with HS256 and checked for validity and expiry. The filter also verifies the account is active and matches its claims. A signed token payload is readable and should not contain a password.

16 Why are frontend route checks insufficient

A person can call an API without using the interface. ProtectedRoute and RoleRoute help navigation, while SecurityConfig and ownership checks enforce the actual API permissions.

17 Can one customer open another request

The customer lookup includes both request ID and the authenticated customer ID. A record outside that ownership scope returns 404. The request body is not trusted to choose a different owner.

18 What is the difference between stock ledger and audit

The inventory ledger explains a quantity movement using before/after balances and references. Sale audit history explains a user action such as creation, editing or cancellation. One business action may generate both.

19 How have you tested the project

Explain only checks you actually performed. The repository contains unit, controller, security and opt in PostgreSQL regression tests. Show relevant test source and your latest execution results; a test file’s existence alone does not show that it passed.

20 What was your contribution and what is next

Describe the modules you personally implemented or can substantiate, then trace their code and demonstrate them. Do not present a memorized invented contribution. For future scope, explain a concrete addition such as converting an approved request into an order with separate stock and payment rules.

14 Demo plan and final revision

A five to seven minute demonstration

- Introduce the problem and users, then show the public storefront. State that custom requests and counter sales are separate workflows.
- As a customer, change a bag color, size and one supported option. Rotate the 3D view, explain the estimate, submit a request and open its saved detail.
- As ADMIN, open the same request, move it to REVIEWING and add a note. Return as the customer and refresh to show the response.
- As an inventory manager, show a product balance, receive a small purchase and open its STOCK_IN movement. Use known demo data.
- As a cashier, sell one unit and show the balance decrease. Explain subtotal and discount.
- As a manager/admin, show a report with its date filters, then cancel the demo sale and show the restored stock, cancellation reason and audit. Finish by demonstrating a safe rejected action, such as an unsupported request transition.

Code reading map

Backend paths below are relative to src/main/java/com/inventory/inventorymanagement/. Frontend paths are relative to frontend/src/.

Topic	Files to open
Entry and navigation	InventoryManagementApplication.java; frontend main.tsx and App.tsx
HTTP and cache	frontend api/axiosClient.ts; features/products/useProducts.ts
Security	security/SecurityConfig.java; JwtAuthenticationFilter.java; JwtTokenService.java
Stock and sales	service/impl/SaleServiceImpl.java; PurchaseServiceImpl.java; repository/ProductRepository.java
Custom requests	service/impl/CustomBagRequestServiceImpl.java; service/CustomBagPricingService.java
3D preview	frontend features/custom-bag/three/BagPreview3D.tsx and bagConfiguration.ts
Dependencies and schema	Project pom.xml; frontend/package.json; src/main/resources/db/migration/
Tests	src/test/java/com/inventory/inventorymanagement/; README.md verification section

Before entering the oral

Be able to explain: the project in one minute; one request from screen to database; the job of each major library; sale and purchase stock direction; transactions and row locks; JWT and ownership; 3D configuration; server pricing; request status transitions; and one limitation. Keep the application, relevant code and genuine test results ready.

When uncertain, describe the rule you can verify and open its implementation. A clear explanation of one real workflow is stronger than a list of library names.